Week3 Day1 Assignment:

Generate Automation code for UiBank registration Page :

Link :https://uibank.uipath.com/register-account

Selenium Java code :

Code GeneratorSettings
Feature File Page Class
Supports: Java/Selenium, TypeScript/Playwright, and Cucumber features
Reset Chat

```
Feature: User Registration on UI Bank

  As a new user I want to register an account so that I can access the UI
Bank services.

  Scenario Outline: Successful registration with valid data
    Given I open the registration page
    When I type "<email>" into the Email field
    And I type "<password>" into the Password field
    And I type "<firstName>" into the First Name field
    And I type "<lastName>" into the Last Name field
    And I type "<middleName>" into the Middle Name field
    And I select "<sex>" from the Sex dropdown
    And I select "<title>" from the Title dropdown
    And I select "<employmentStatus>" from the Employment Status dropdown
    And I type "<dob>" into the Date of Birth field
    And I select "<maritalStatus>" from the Marital Status dropdown
    And I type "<dependents>" into the Number of Dependents field
    And I type "<username>" into the Username field
    And I check the Agree to Privacy Policy checkbox
    And I click the Register button
    Then I should see a registration success message

    Examples: South-India realistic data
      | email                     | password   | firstName | lastName |
middleName | sex    | title | employmentStatus | dob       | maritalStatus
| dependents | username      |
      | ramesh.kumar@example.com  | Pass@1234  | Ramesh    | Kumar    | K
| Male   | Ms    | Full-time        | 12/05/90  | Single        | 0
| rameshk90     |
      | lakshmi.nair@example.com  | Secure!56  | Lakshmi   | Nair     | L
| Female | Mrs   | Part-time        | 23/08/88  | Married       | 2
| lakshmian88   |
      | vijay.srinivas@example.in | Vj@2023!   | Vijay     | Srinivas | V
| Male   | Ms    | Full-time        | 07/11/92  | Single        | 1
| vijays92      |
```

```java
package com.uibank.stepdefs;

import io.cucumber.java.After;
import io.cucumber.java.Before;
import io.cucumber.java.en.*;

import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.*;

import java.time.Duration;

public class RegistrationStepDefinitions {

    private WebDriver driver;
    private WebDriverWait wait;

    @Before
    public void setUp() {
        // Assuming chromedriver is on the system PATH
        driver = new ChromeDriver();
        wait = new WebDriverWait(driver, Duration.ofSeconds(15));
        driver.manage().window().maximize();
    }

    @After
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }

    @Given("I open the registration page")
    public void openRegistrationPage() {
        driver.get("https://uibank.uipath.com/register-account");
        // Wait for the form to be present

wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("email")));
    }

    @When("I type {string} into the Email field")
    public void typeEmail(String email) {
        WebElement el =
wait.until(ExpectedConditions.elementToBeClickable(By.id("email")));
        el.clear();
        el.sendKeys(email);
    }

    @When("I type {string} into the Password field")
    public void typePassword(String password) {
        WebElement el =
wait.until(ExpectedConditions.elementToBeClickable(By.id("password")));
        el.clear();
```

```
        el.sendKeys(password);
    }

    @When("I type {string} into the First Name field")
    public void typeFirstName(String firstName) {
        WebElement el =
wait.until(ExpectedConditions.elementToBeClickable(By.id("firstName")));
        el.clear();
        el.sendKeys(firstName);
    }

    @When("I type {string} into the Last Name field")
    public void typeLastName(String lastName) {
        WebElement el =
wait.until(ExpectedConditions.elementToBeClickable(By.id("lastName")));
        el.clear();
        el.sendKeys(lastName);
    }

    @When("I type {string} into the Middle Name field")
    public void typeMiddleName(String middleName) {
        WebElement el =
wait.until(ExpectedConditions.elementToBeClickable(By.id("middleName")));
        el.clear();
        el.sendKeys(middleName);
    }

    @When("I select {string} from the Sex dropdown")
    public void selectSex(String sex) {
        WebElement dropdown =
wait.until(ExpectedConditions.elementToBeClickable(By.id("sex")));
        new Select(dropdown).selectByVisibleText(sex);
    }

    @When("I select {string} from the Title dropdown")
    public void selectTitle(String title) {
        WebElement dropdown =
wait.until(ExpectedConditions.elementToBeClickable(By.id("title")));
        new Select(dropdown).selectByVisibleText(title);
    }

    @When("I select {string} from the Employment Status dropdown")
    public void selectEmploymentStatus(String status) {
        WebElement dropdown =
wait.until(ExpectedConditions.elementToBeClickable(By.id("employmentStatus"
)));
        new Select(dropdown).selectByVisibleText(status);
    }

    @When("I type {string} into the Date of Birth field")
    public void typeDOB(String dob) {
        WebElement el =
wait.until(ExpectedConditions.elementToBeClickable(By.id("age")));
```

```
        el.clear();
        el.sendKeys(dob);
    }

    @When("I select {string} from the Marital Status dropdown")
    public void selectMaritalStatus(String status) {
        WebElement dropdown =
wait.until(ExpectedConditions.elementToBeClickable(By.id("maritalStatus")))
;
        new Select(dropdown).selectByVisibleText(status);
    }

    @When("I type {string} into the Number of Dependents field")
    public void typeDependents(String dependents) {
        WebElement el =
wait.until(ExpectedConditions.elementToBeClickable(By.id("numberOfDependent
s")));
        el.clear();
        el.sendKeys(dependents);
    }

    @When("I type {string} into the Username field")
    public void typeUsername(String username) {
        WebElement el =
wait.until(ExpectedConditions.elementToBeClickable(By.id("username")));
        el.clear();
        el.sendKeys(username);
    }

    @When("I check the Agree to Privacy Policy checkbox")
    public void checkAgreeCheckbox() {
        WebElement checkbox =
wait.until(ExpectedConditions.elementToBeClickable(By.id("agreeCheckbox")))
;
        if (!checkbox.isSelected()) {
            checkbox.click();
        }
    }

    @When("I click the Register button")
    public void clickRegister() {
        // The button becomes enabled only after the checkbox is ticked
        WebElement btn =
wait.until(ExpectedConditions.elementToBeClickable(
                By.xpath("//button[normalize-space()='Register']")));
        btn.click();
    }

    @Then("I should see a registration success message")
    public void verifySuccessMessage() {
        // Assuming the application shows a div with class 'alert-success'
after registration
        WebElement successMsg = wait.until(
```

```
ExpectedConditions.visibilityOfElementLocated(By.cssSelector(".alert-
success")));
        if (!successMsg.isDisplayed()) {
            throw new AssertionError("Success message not displayed");
        }
    }
}
```

Copy
InspectGenerate

Playwright with Typescript:

```
Feature: User registration on UI Bank

  As a prospective customer from South-India
  I want to register a new account using the registration form
  So that I can later log in to the UI Bank demo application

  Background:
    Given the registration page is open

  Scenario Outline: Register a new user with valid data
    When I fill the registration form with data set "<dataset>"
    And I accept the privacy policy
    And I submit the registration form
    Then I should see a registration success message

    Examples:
      | dataset          |
      | chennai_user     |
      | bengaluru_user   |
      | hyderabad_user   |
```

```
/* eslint-disable @typescript-eslint/no-non-null-assertion */
import { Browser, BrowserContext, chromium, Page } from '@playwright/test';
import {
  Before,
  After,
  Given,
  When,
  Then,
  setWorldConstructor,
  World as CucumberWorld,
} from '@cucumber/cucumber';

/* ---------------------------------------------------------------------
-- */
/*  World definition — holds the Playwright objects and test data
*/
```

```
/* ------------------------------------------------------------------------
-- */
interface World extends CucumberWorld {
  browser?: Browser;
  context?: BrowserContext;
  page?: Page;
  baseUrl: string;
  /** Helper to retrieve a data-set by its key */
  getDataSet(name: string): Record<string, string>;
}

/* ------------------------------------------------------------------------
-- */
/*  Test data – realistic South-India user information
*/
/* ------------------------------------------------------------------------
-- */
const DATA_SETS: Record<string, Record<string, string>> = {
  chennai_user: {
    email: 'ravi.kumar@example.com',
    password: 'P@ssw0rd123',
    firstName: 'Ravi',
    lastName: 'Kumar',
    middleName: 'M',
    sex: 'male',
    title: 'mr',
    employmentStatus: 'Full-time',
    age: '12/05/90',
    maritalStatus: 'Single',
    numberOfDependents: '0',
    username: 'ravi.kumar90',
  },
  bengaluru_user: {
    email: 'shreya.patel@example.com',
    password: 'Str0ng!Pass',
    firstName: 'Shreya',
    lastName: 'Patel',
    middleName: '',
    sex: 'female',
    title: 'ms',
    employmentStatus: 'Part-time',
    age: '08/22/88',
    maritalStatus: 'Married',
    numberOfDependents: '2',
    username: 'shreya.patel88',
  },
  hyderabad_user: {
    email: 'arun.reddy@example.com',
    password: 'Arun#2024',
    firstName: 'Arun',
    lastName: 'Reddy',
    middleName: 'K',
    sex: 'male',
```

```
    title: 'mr',
    employmentStatus: 'Full-time',
    age: '03/15/85',
    maritalStatus: 'Married',
    numberOfDependents: '1',
    username: 'arun.reddy85',
  },
};

/* ---------------------------------------------------------------------
-- */
/*  World implementation
*/
/* ---------------------------------------------------------------------
-- */
class CustomWorld implements World {
  browser?: Browser;
  context?: BrowserContext;
  page?: Page;
  baseUrl = 'https://uibank.uipath.com/register-account';

  getDataSet(name: string) {
    const data = DATA_SETS[name];
    if (!data) {
      throw new Error(`Data set "${name}" is not defined`);
    }
    return data;
  }
}
setWorldConstructor(CustomWorld);

/* ---------------------------------------------------------------------
-- */
/*  Hooks – launch/close browser
*/
/* ---------------------------------------------------------------------
-- */
Before(async function (this: World) {
  this.browser = await chromium.launch({ headless: true });
  this.context = await this.browser.newContext();
  this.page = await this.context.newPage();
});

After(async function (this: World) {
  await this.page?.close();
  await this.context?.close();
  await this.browser?.close();
});

/* ---------------------------------------------------------------------
-- */
/*  Re-usable low-level helpers
*/
```

```
/* ------------------------------------------------------------------------
-- */
async function fillInput(page: Page, selector: string, value: string) {
  const element = page.locator(selector);
  await element.waitFor({ state: 'visible', timeout: 5000 });
  await element.fill(value);
}

async function selectOption(page: Page, selector: string, value: string) {
  const element = page.locator(selector);
  await element.waitFor({ state: 'visible', timeout: 5000 });
  await element.selectOption({ value });
}

/* ------------------------------------------------------------------------
-- */
/*  Step definitions
*/
/* ------------------------------------------------------------------------
-- */
Given('the registration page is open', async function (this: World) {
  try {
    await this.page!.goto(this.baseUrl, { waitUntil: 'networkidle' });
    // Ensure the form is present before proceeding
    await this.page!.locator('form.form-box').waitFor({ state: 'visible',
timeout: 8000 });
  } catch (err) {
    throw new Error(`Failed to open registration page: ${(err as
Error).message}`);
  }
});

When(
  'I fill the registration form with data set {string}',
  async function (this: World, datasetName: string) {
    const data = this.getDataSet(datasetName);
    const p = this.page!;

    try {
      // ---- Credentials ----------------------------------------------
---
      await fillInput(p, 'input#email', data.email);
      await fillInput(p, 'input#password', data.password);

      // ---- Personal details -----------------------------------------
----
      await fillInput(p, 'input#firstName', data.firstName);
      await fillInput(p, 'input#lastName', data.lastName);
      await fillInput(p, 'input#middleName', data.middleName);

      // ---- Dropdowns ------------------------------------------------
----
      await selectOption(p, 'select#sex', data.sex);
```

```
      await selectOption(p, 'select#title', data.title);
      await selectOption(p, 'select#employmentStatus',
data.employmentStatus);
      await selectOption(p, 'select#maritalStatus', data.maritalStatus);

      // ---- Remaining fields ----------------------------------------
----
      await fillInput(p, 'input#age', data.age);
      await fillInput(p, 'input#numberOfDependents',
data.numberOfDependents);
      await fillInput(p, 'input#username', data.username);

      // Wait for the Register button to become enabled (all required
fields filled)
      const registerBtn = p.locator('button:has-text("Register")');
      await registerBtn.waitFor({ state: 'attached', timeout: 5000 });
      await expect(registerBtn).toBeEnabled({ timeout: 8000 });
    } catch (err) {
      throw new Error(`Error while filling the form for dataset
"${datasetName}": ${(err as Error).message}`);
    }
  },
);

When('I accept the privacy policy', async function (this: World) {
  const p = this.page!;
  try {
    const checkbox = p.locator('#agreeCheckbox');
    await checkbox.waitFor({ state: 'visible', timeout: 5000 });
    const isChecked = await checkbox.isChecked();
    if (!isChecked) {
      await checkbox.check();
    }
  } catch (err) {
    throw new Error(`Unable to check the privacy-policy checkbox: ${(err as
Error).message}`);
  }
});

When('I submit the registration form', async function (this: World) {
  const p = this.page!;
  try {
    const submitBtn = p.locator('button:has-text("Register")');
    await submitBtn.waitFor({ state: 'visible', timeout: 5000 });
    await submitBtn.click();

    // The demo site shows a toast with class .toast-message — wait for it
    await p.locator('.toast-message').waitFor({ state: 'visible', timeout:
10000 });
  } catch (err) {
    throw new Error(`Form submission failed: ${(err as Error).message}`);
  }
});
```

```
Then('I should see a registration success message', async function (this:
World) {
  const p = this.page!;
  try {
    const successMsg = p.locator('.toast-message', { hasText: /registration
successful/i });
    await successMsg.waitFor({ state: 'visible', timeout: 8000 });
    const isVisible = await successMsg.isVisible();
    if (!isVisible) {
      throw new Error('Success toast is not visible');
    }
  } catch (err) {
    throw new Error(`Success verification failed: ${(err as
Error).message}`);
  }
});
```